NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

Nitte (DU) established under Section 3 of UGC Act 1956 | Accredited with 'A+' Grade by NAAC

Department of Computer Science & Engineering

Report on Mini Project

AptiTest – Dynamic Aptitude Test Management and Automated Evaluation System

Course Code: CS3603-1

Course Name: Java and Spring Framework Lab

Semester: V SEM Section: B

Academic Year: 2026–2027

Submitted To:

Mr. Akhilraj V. Gadagkar

Assistant Professor Grade - II

Department of Computer Science and Engineering

NMAM Institute of Technology, Nitte

Submitted By:

Jeevan SB (NNM24CS105)

MK Subramanya (NNM24CS124)

Mayur D Shetty (NNM24CS136)

Eashan S Gaikwaad (NNM24CS084)

Date of submission:

12/11/2026

Signature of the Course Instructor

NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

CERTIFICATE

*This is to certify that the Mini-Project report titled "**AptiTest – Dynamic Aptitude Test Management and Automated Evaluation System**" has been submitted by **Mr. Jeevan SB (NNM24CS105), Mr. MK Subramanya (NNM24CS124), Mr. Mayur D Shetty (NNM24CS136) and Mr. Eashan S Gaikwaad (NNM24CS084)**. The above students of V SEM B.Tech., Section B, are bonafide students of NMAM Institute of Technology, Nitte, and have successfully completed the Mini-Project during **2026–2027** in **Computer Science and Engineering** at NMAM Institute of Technology, Nitte.*

_____	_____
Name and Signature of the Course Instructor	*Signature of the HOD*

ABSTRACT

AptiTest – Dynamic Aptitude Test Management and Automated Evaluation System is a web-based mini-project designed to support structured aptitude and technical practice for Computer Science and Information Technology students. It integrates question administration, configurable test generation, timed answering, automated evaluation and progress review within a single role-based application.

The implementation targets Java 17 and uses Spring Boot 3.5.16, Spring MVC, JSP/JSTL, Spring Security and Spring Data JPA. Persistent H2 is the default database, with an optional MySQL profile. Administrators manage categorized questions, review pending content, configure test templates and inspect student progress. Students register and verify accounts, choose practice settings, answer questions with saved state and review completed results and explanations.

Test generation filters approved questions by template settings and selected categories, optionally excludes previously viewed items and randomizes a unique subset. Per-attempt snapshots preserve the question text, accepted answers and grading rules. A server deadline, transactional state updates and idempotent submission govern the attempt lifecycle. Ten answer formats are evaluated using explicit rules for choices, numerical tolerance and units, fractions, ratios, booleans and normalized text. Revision attempts remain reviewable but are excluded from cumulative rankings and statistics.

The inspected seed file contains 238 questions, comprising 187 active and 51 pending items, and the bootstrap defines five templates. This report combines source inspection, independently recounted seed data and genuine repository screenshots. The repository documents 61 Java cases and 47 packaged HTTP acceptance checks; these results are attributed to that verification record and were not rerun for this report. External MySQL connectivity, provider delivery/live payments and production-scale performance are not established. The project provides an educational practice foundation, while content expansion, operational hardening and integration validation remain future work.

Keywords: aptitude practice, Spring MVC, JSP, automated evaluation, question snapshots, role-based access.

TABLE OF CONTENTS

Title Page....i

Institution Certificate....ii

Abstract....iii

Table of Contents....iv

1 INTRODUCTION....1

1.1 Background and motivation....1

1.2 Project overview....1

1.3 Scope and principal users....2

1.4 Report basis and boundaries....2

2 PROBLEM STATEMENT....3

2.1 Identified problem....3

2.2 Operational constraints....3

3 OBJECTIVES....4

3.1 General objective....4

3.2 Specific objectives....4

3.3 Evaluation approach....4

4 TECHNOLOGIES USED....5

4.1 Verified application stack....5

4.2 Browser and verification tools....5

4.3 Persistence and deployment configuration....6

4.4 Build and configuration rationale....6

5 IMPLEMENTATION....7

5.1 System architecture....7

5.2 Layers, modules and request flow....8

5.3 Entity–relationship model....9

5.4 Data dictionary and persistence rules....10

5.5 Registration, authentication and profile management....11

5.6 Question bank, review and seeded content....12
5.7 Test templates, filtering and randomization....13
5.8 End-to-end test-taking workflow....14
5.9 Timer, autosave, navigation and resume....15
5.10 Automated evaluation rules....16
5.11 Results, history, revision and analytics....18
5.12 Administrator workflows....19
5.13 Payment access and external-service scope....20
5.14 Security, validation and deployment boundaries....21
6 RESULTS AND DISCUSSION....22
6.1 Evidence and verification scope....22
6.2 Authentication interface....23
6.3 Student dashboard and test selection....24
6.4 Timed practice and persisted responses....25
6.5 Result review and interpretation....26
6.6 Administrator overview....27
6.7 Responsive presentation and screenshot provenance....28
6.8 Discussion, reproducibility and limitations....29
7 CONCLUSION AND FUTURE SCOPE....30
7.1 Conclusion....30
7.2 Learning outcomes....30
7.3 Prioritized future scope....31
REFERENCES....32

1 INTRODUCTION

1.1 Background and motivation

Aptitude and technical assessments form an important part of placement preparation for Computer Science and Information Technology students. Regular practice involves more than answering a fixed question paper: learners need suitable questions, controlled practice time, consistent evaluation and a useful record of their mistakes. Instructors also need a maintainable question bank and a practical way to publish tests without repeatedly preparing and checking individual papers.

Paper-based practice and disconnected online forms make these activities difficult to coordinate. A learner may repeatedly encounter familiar questions, lose an unfinished response after refreshing a page, or see only a total mark without an explanation. An administrator may have to maintain question content, test configuration and student progress in unrelated files. These limitations motivate an integrated web application with a clearly defined assessment lifecycle.

1.2 Project overview

AptiTest – Dynamic Aptitude Test Management and Automated Evaluation System is a Java and Spring Framework mini-project that brings question administration, test generation, timed practice, automated evaluation and progress review into one application. Students use a dedicated practice workspace, while administrators manage approved content, test templates and student accounts. The interface is rendered on the server using JSP and JSTL rather than a separate single-page frontend.

The application follows a layered Spring MVC structure. Controllers handle web requests; services enforce account, question, attempt and grading rules; Spring Data JPA repositories persist the domain model. The inspected source targets Java 17 and uses Spring Boot 3.5.16 with executable WAR packaging. Persistent file-based H2 is the default database, and a separate MySQL profile is available. These details are taken from the source repository, not inferred from the appearance of its screens [1].

1.3 Scope and principal users

Table 1.1 Actors and their implemented responsibilities

Actor	Responsibilities
Student	Register and verify an account; sign in; update profile/password; select a test; answer and save; submit; inspect results, history, solved questions and rankings.
Administrator	Create/edit/review questions; import JSON; configure/archive templates; inspect student summaries and completed attempts; enable or disable student accounts.
Scheduled worker	Find expired in-progress attempts and invoke the same server-side submission logic used by interactive actions.

The core scope is educational practice. It includes configurable tests, ten supported answer types, saved responses, server-enforced expiry, revision attempts and statistics. A fresh test may exclude questions previously viewed by the same student, while an explicit option permits repeats when required. Existing attempts retain question snapshots, so subsequent edits to the live question bank do not rewrite their historical evaluation.

1.4 Report basis and boundaries

This report describes the public Aptitest-JSF repository at revision d23b6e798b4c82222642ab03a7ba9f8f1b72163c [1]. The POM, configuration, controllers, services, entities, repositories, JSP views, JavaScript, seed data and test sources were inspected. The seed counts were independently recomputed from the supplied JSON. The repository’s verification document is cited separately for previously executed application tests [2]; those Java and HTTP suites were not rerun during preparation of this report.

The system is not a proctored examination platform and does not claim anti-cheating controls, adaptive learning, live payment certification or production-scale performance. Default demonstration mail and checkout behaviour is described explicitly. The institutional report template supplies the academic layout; its sample identities and empty chapter pages are replaced with the present project material.

2 PROBLEM STATEMENT

2.1 Identified problem

Students require a consistent environment in which aptitude and CSE/IT fundamentals can be practised, evaluated and reviewed. In a manually coordinated process, test creation, answer collection, marking and progress tracking are separate activities. This increases repeated administrative work and makes it harder for a learner to identify which topics require further practice.

The problem addressed is therefore to design and implement a role-based web application that creates a suitable timed test from approved questions, preserves responses during the attempt, applies type-specific grading rules and presents a reliable record of the outcome. The administrator must retain control over question quality and test configuration without compromising completed attempts.

Table 2.1 Requirements derived from the problem

Problem	Required response
Static or repetitive papers	Filter approved questions by template and topic, randomize selection and optionally exclude previously viewed questions.
Inconsistent manual marking	Use an explicit grading dispatcher for MCQ, numerical, fractional, ratio, boolean and short-text formats.
Lost or ambiguous attempt state	Persist answers and flags; resume an existing active attempt; use a server deadline.
Weak review and accountability	Store question snapshots, results and explanations; distinguish fresh practice from revision.
Uncontrolled content changes	Separate administrator routes, pending review and archival from student practice.

2.2 Operational constraints

Question availability must be checked after every filter, including the student's seen-question history. A test must not be partially created when the pool is too small. Ownership, role and deadline checks must be performed on the server, not only through disabled controls. The implementation also has to remain deployable as a relatively simple Java web application suitable for a laboratory mini-project.

3 OBJECTIVES

3.1 General objective

To develop a maintainable, database-backed aptitude practice platform that integrates dynamic test generation, controlled test-taking and automated evaluation for CSE and IT students using Java and the Spring Framework.

3.2 Specific objectives

- Provide registration, email verification, form-based login and separate student/administrator workspaces, with profile and password-management facilities.
- Maintain categorized questions with difficulty, answer type, options, accepted answers, grading configuration, solutions and review status.
- Allow administrators to configure test size, duration, difficulty, content filters, reattempt policy and optional paid access.
- Generate a unique randomized subset from the eligible active pool and support a student-controlled previously-seen setting.
- Persist responses and review flags, resume active attempts and enforce expiry using server time.
- Evaluate supported answer formats consistently, preserve historical question snapshots and present correct, incorrect and skipped outcomes.
- Offer history, solved-question review, topic performance and rankings without allowing revision attempts to inflate cumulative statistics.

3.3 Evaluation approach

The objectives are assessed through traceability to source modules, inspection of the seed data, genuine application captures and the repository's documented verification. Chapter 6 distinguishes these evidence types. No target for response time, concurrent users, learning improvement or external-provider success is asserted because corresponding measurements were not supplied or independently executed.

4 TECHNOLOGIES USED

4.1 Verified application stack

Table 4.1 Technologies declared by the inspected source

Technology	Role and verified configuration
Java 17	POM language target; domain, service and controller implementation.
Spring Boot 3.5.16	Parent POM, dependency management, application startup and executable WAR packaging.
Spring MVC	Annotated controllers, form/model binding, redirects, validation errors and JSP view resolution.
JSP and JSTL	Server-rendered views under WEB-INF/views; reusable tags, escaped content, loops and conditional UI.
Spring Security	Session/form login, route authorization, CSRF protection, BCrypt and account-state filtering.
Spring Data JPA / Hibernate	Repository interfaces, entity mapping, transactions, foreign keys, optimistic versions and row locks.
H2 / MySQL Connector/J	File-based H2 by default; MySQL configured through an optional Spring profile.
Maven / Maven Wrapper	Dependency resolution, testing and WAR packaging; wrapper selects Maven 3.9.11.

The POM includes web, JPA, security and validation starters; Tomcat and Jasper are provided dependencies appropriate to WAR packaging. Both the Jakarta JSTL API and its implementation are declared. Database driver dependencies are runtime-scoped. Managed transitive versions are controlled by the Spring Boot parent; this report does not invent independently pinned versions for them [1, 4–7].

4.2 Browser and verification tools

The frontend uses HTML, CSS and plain JavaScript. JavaScript enhances autosave, navigation, countdown, theme preference and optional gateway checkout. JUnit Jupiter, Spring Boot Test, Spring Security Test and MockMvc appear in the Java test sources. A Python HTTP smoke utility exercises the packaged WAR; Node is useful for JavaScript syntax checks but is not required to run the Java application.

4.3 Persistence and deployment configuration

The default datasource is a persistent H2 file at ./data/aptitest, not an in-memory database. JPA schema handling is configured as update and open-in-view is disabled. The application exposes its MVC views through /WEB-INF/views/ with the .jsp suffix. The server binds to 0.0.0.0 and defaults to port 8080, with environment-variable overrides. Timestamps are persisted with a UTC JDBC setting and student-facing dates are formatted for Asia/Kolkata.

Table 4.2 Default run compared with the optional MySQL profile

Setting	Default configuration	MySQL profile
Database	File-based H2	MySQL URL and application credentials via environment
Seed / demonstration mode	Enabled by default	Disabled by default
Mail mode	In-application demonstration links	SendGrid adapter selected; credentials required
Payment mode	Labelled no-charge demonstration checkout	Disabled by default; Razorpay can be explicitly configured
Session cookie	HttpOnly; SameSite=Lax	Also defaults to Secure for HTTPS deployment
Schema handling	Hibernate update	Inherited update; versioned production migrations remain future work

4.4 Build and configuration rationale

WAR packaging supports JSP compilation and both executable and servlet-container entry points. The normal commands are ./mvnw clean verify followed by java -jar target/aptitest.war. Selecting --spring.profiles.active=mysql uses the separate profile; database, administrator, base-URL, mail and gateway settings are supplied externally. Secrets are not report content and must not be embedded in screenshots or shared configuration.

The default self-contained configuration lowers laboratory setup requirements, but it is not a production configuration. The profile does not by itself establish tested MySQL connectivity, schema migration safety or successful delivery/capture by external providers. Those deployment checks remain separate from the source inspection and documented H2-based verification [2, 3].

5 IMPLEMENTATION

5.1 System architecture

AptiTest is a layered, server-rendered monolithic web application. Figure 5.1 shows the request boundary, application logic and persistence layers. Optional mail and payment adapters make outbound calls to SendGrid and Razorpay; they are integrations of the same application, not separate assessment microservices.

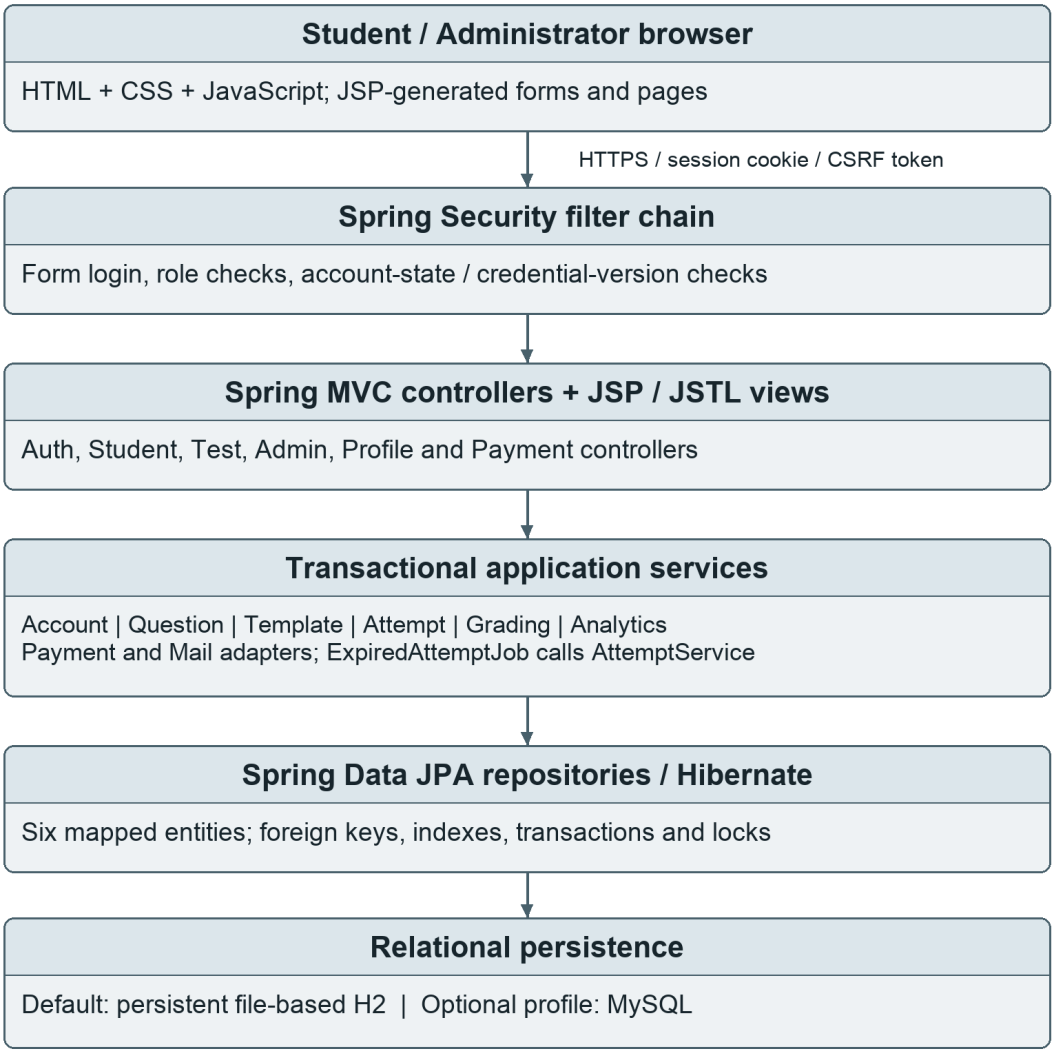


Figure 5.1 Layered AptiTest architecture reconstructed from source.

Source: SecurityConfig, MVC controllers, service classes, JPA repositories and application properties [1]. H2 is the default; MySQL is optional.

5.2 Layers, modules and request flow

Table 5.1 Principal module responsibilities

Layer / module	Source responsibility
Configuration / security	SecurityConfig, AppPrincipal and AccountStateFilter authenticate, authorize and revoke stale account sessions.
Web / views	Auth, Student, Test, Admin, Profile, Payment and Site controllers; JSP/JSTL render forms, tables and notices.
Account / mail	AccountService validates credentials and token lifecycles; MailService supplies demonstration or SendGrid delivery.
Content / templates	QuestionService validates/reviews/imports content; TemplateService filters eligible question pools and saves policies.
Attempt / grading	AttemptService controls snapshots, saved state, ownership, deadlines and submission; GradingService compares answers.
Analytics / payments	AnalyticsService computes student/admin summaries; PaymentService manages template access and optional provider verification.
Persistence / worker	Six entity/repository pairs persist state; ExpiredAttemptJob finalizes expired attempts through the attempt service.

5.2.1 Example: saving one response

A student submits an answer form or autosave request with the session cookie and CSRF token. Spring Security first checks authentication and the student route. TestController obtains the authenticated user ID from AppPrincipal rather than trusting a user ID in the form. AttemptService locks the attempt, checks ownership and expiry, resolves the question position and validates the submitted value. It then updates the answer and review flag transactionally. The controller returns either a redirect or a small JSON status, and the browser updates its save indicator.

This separation is important: the controller coordinates HTTP interaction, but the service enforces the rules shared by normal forms, asynchronous saves and scheduled expiry. Repository calls are behind these service checks rather than exposing unrestricted entity JSON endpoints.

5.3 Entity–relationship model

Figure 5.2 represents the six JPA entities and their declared references. Scalar foreign-key IDs are also mapped through read-only lazy associations for database integrity. Arrows run from a referenced entity to dependent rows; endpoint labels show relationship multiplicity.

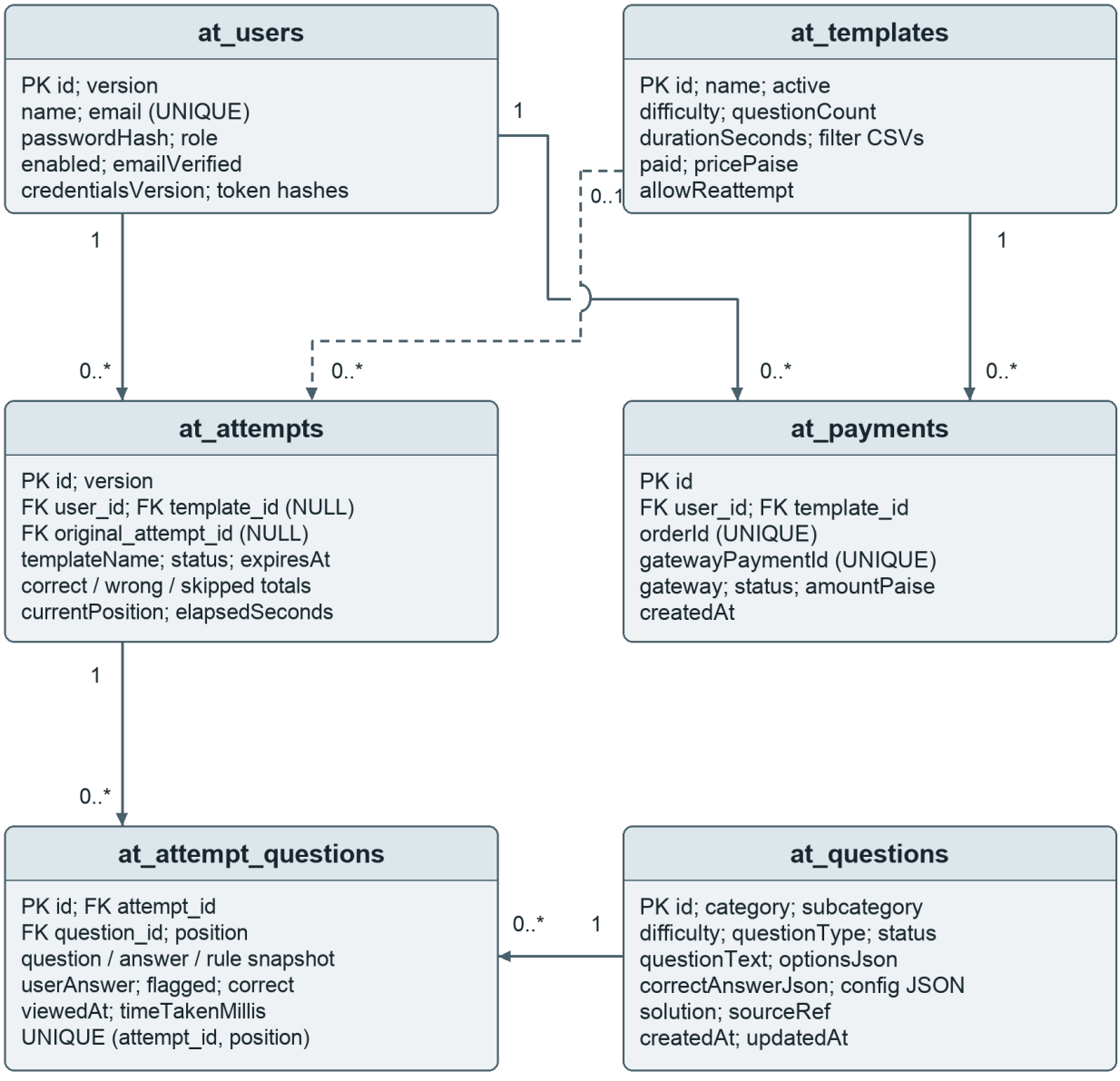


Figure 5.2 Source-derived entity relationships and principal keys.

Solid lines: required references. Dashed line: an attempt may have no template reference. The optional original-attempt self-reference supports revision history.

5.4 Data dictionary and persistence rules

Table 5.2 Entity purpose and important stored data

Entity / table	Purpose and principal data
AppUser / at_users	Unique email, name, password hash, role, enabled/verified flags, credential version, token hashes/expiries and optimistic version.
Question / at_questions	Category/subcategory, difficulty/type, text, passage/data, JSON options/answer/configuration, solution, status and source reference.
TestTemplate / at_templates	Name, question count, duration, CSV content filters, price in paise, active and reattempt policies.
TestAttempt / at_attempts	User/template/original-attempt IDs, copied template name/settings, lifecycle times, status, counts, elapsed time and current position.
AttemptQuestion / at_attempt_questions	Question and attempt IDs, ordered position, frozen content/answer/rules/solution, response, flag, correctness, first view and accumulated time.
PaymentRecord / at_payments	User/template IDs, unique order/payment references, gateway, status, amount in paise and creation time.

An attempt contains many AttemptQuestion rows, but each position is unique within that attempt. Foreign keys link these rows to their attempt and original question. Attempts link to a user, an optional template and an optional original attempt. Payments require both a user and a template. Unique email and order/payment references, plus indexes on question filtering, attempt ownership/expiry and payment access, support the principal lookups.

Users and attempts have @Version fields; selected user/attempt repository queries also use pessimistic write locks. These mechanisms serve different purposes: versions detect conflicting updates, while locks serialize critical start/save/submit operations. Template archival preserves rows needed by past attempts and payments. Question snapshots preserve historical evaluation data, although referential integrity still prevents deletion of a referenced live question.

Schema generation uses Hibernate update in the inspected configuration. Controlled, versioned production migration and backup/restore verification are not implemented by these annotations.

5.5 Registration, authentication and profile management

SignupForm validates the student's name, email and password fields. AccountService normalizes the email, rejects an existing address, checks password confirmation and always creates a STUDENT role. Extra request parameters cannot choose an administrator role. The password is stored using BCrypt with cost 12; the service enforces at least eight characters and at most 72 UTF-8 bytes.

A registration token is generated from 32 SecureRandom bytes. Only its SHA-256 hash and expiry are stored on the user row. Verification links expire after 24 hours and are consumed on success. Password-recovery tokens use the same hashed-token pattern with a 30-minute expiry. Unknown, disabled or unverified reset requests do not disclose a usable account token.

Table 5.3 Account lifecycle and enforcement

Operation	Server behaviour
Sign in	Spring Security form login uses the email parameter; students redirect to /dashboard and administrators to /admin.
Verify / reset	Hash lookup, expiry check and single-use token consumption; reset changes the password hash and credential version.
Profile name	Authenticated account only; trimmed name of 1–100 characters.
Change password	Check current password and confirmation; change credentials and invalidate the active session.
Disable student	Administrator changes enabled status and credential version; the account-state filter rejects old sessions.

AppPrincipal records identity, authorities and the credential version at login. On subsequent requests AccountStateFilter reloads the account and invalidates the session if it is missing, disabled, unverified or has a different credential version. Thus account revocation is not limited to blocking the next login. Logout is a state-changing POST protected by CSRF.

Default demonstration mode exposes verification previews and an administrator-only in-memory mail outbox. The optional SendGrid path sends account links to a configured sender/provider. Neither the preview nor the existence of the adapter proves real email delivery [3].

5.6 Question bank, review and seeded content

A question stores its category, subcategory, difficulty, type, prompt, optional passage/data block, options, accepted answer, grading configuration and explanation. Status values are pending, active, inactive and rejected. Only active questions enter a new test pool. Pending items are reviewed by an administrator, and approval reruns validation so an incomplete flagged MCQ cannot be activated unchanged.

Table 5.4 Independently counted seed questions by answer type

Question type	All seed rows	Question type	All seed rows
numeric	134	mcq_single	71
ratio	11	fraction	9
boolean	5	mcq_multiple	2
fill_in_blank	2	short_text	2
numeric_with_unit	1	data_interpretation	1
TOTAL	238	Active / pending	187 / 51

The 187 active rows comprise 105 basic, 50 intermediate and 32 advanced questions. Active categories contain 97 Quantitative Aptitude, 30 Logical Reasoning, 39 CSE Fundamentals, 20 Programming Concepts and one Verbal Ability question. The limited verbal pool is therefore a real content limitation, not a broad-coverage claim. Counts describe the supplied seed file before any administrator changes.

The repository identifies 49 approved original items, 51 original items retained for review and 138 supplemental seeded items [1]. SeedDataLoader uses source references to avoid reinserting an existing seed question. A fresh run creates five test templates and public demonstration accounts only when seeding is enabled; non-demonstration startup rejects demonstration mail/payment/seeding settings.

5.6.1 Import and search

JSON import accepts an array of at most 500 questions, validates each row and assigns pending status. The import is transactional: an invalid row aborts the entire batch. The configured multipart size limit is 2 MB. Question search is case-insensitive, status/category-aware and paginated; SQL LIKE wildcard characters are escaped so %, _ and backslash are treated literally.

5.7 Test templates, filtering and randomization

Table 5.5 Five templates created by SeedDataLoader

Template	Questions	Minutes	Difficulty / access
Quick Warm-up	10	10	Mixed / free
Easy Practice - 30 Qs	30	30	Basic / free
Easy Practice - 60 Qs	60	60	Basic / free
Hard Practice - 30 Qs	30	30	Intermediate + advanced / free
Hard Practice - 60 Qs	60	60	Intermediate + advanced / ₹50.00

TemplateForm limits question count to 1–100 and duration to 1–180 minutes. Administrators can set categories, subcategories and answer types as CSV filters, choose difficulty, enable/disable a template, control revision and set a paid price. A paid template must have a strictly positive price; prices are persisted as integer paise. Archiving sets active=false rather than deleting related history.

TemplateService.pool begins with active questions and intersects the template filters with the student's selected categories. Easy means basic; hard combines intermediate and advanced; mixed imposes no difficulty restriction. The full active bank provides 105 easy and 82 hard questions, sufficient for each 60-question preset before personalized exclusions. The dashboard's available count is this base filtered pool, not a promise that the student's unseen pool is large enough.

5.7.1 Selection procedure

- Lock the student row and resume an existing non-expired active attempt if one exists.
- Validate the requested template and required paid access, then build the filtered active pool.
- Unless includeSeen is true, remove question IDs with a non-null viewedAt in this student's prior attempt items.
- Reject insufficient availability; otherwise shuffle with SecureRandom and select the required unique subset.
- Persist the attempt and ordered immutable question snapshots in the same transaction.

"Seen" means actually viewed, not merely allocated or correctly answered. No adaptive difficulty or guaranteed equal topic quotas are implemented.

5.8 End-to-end test-taking workflow

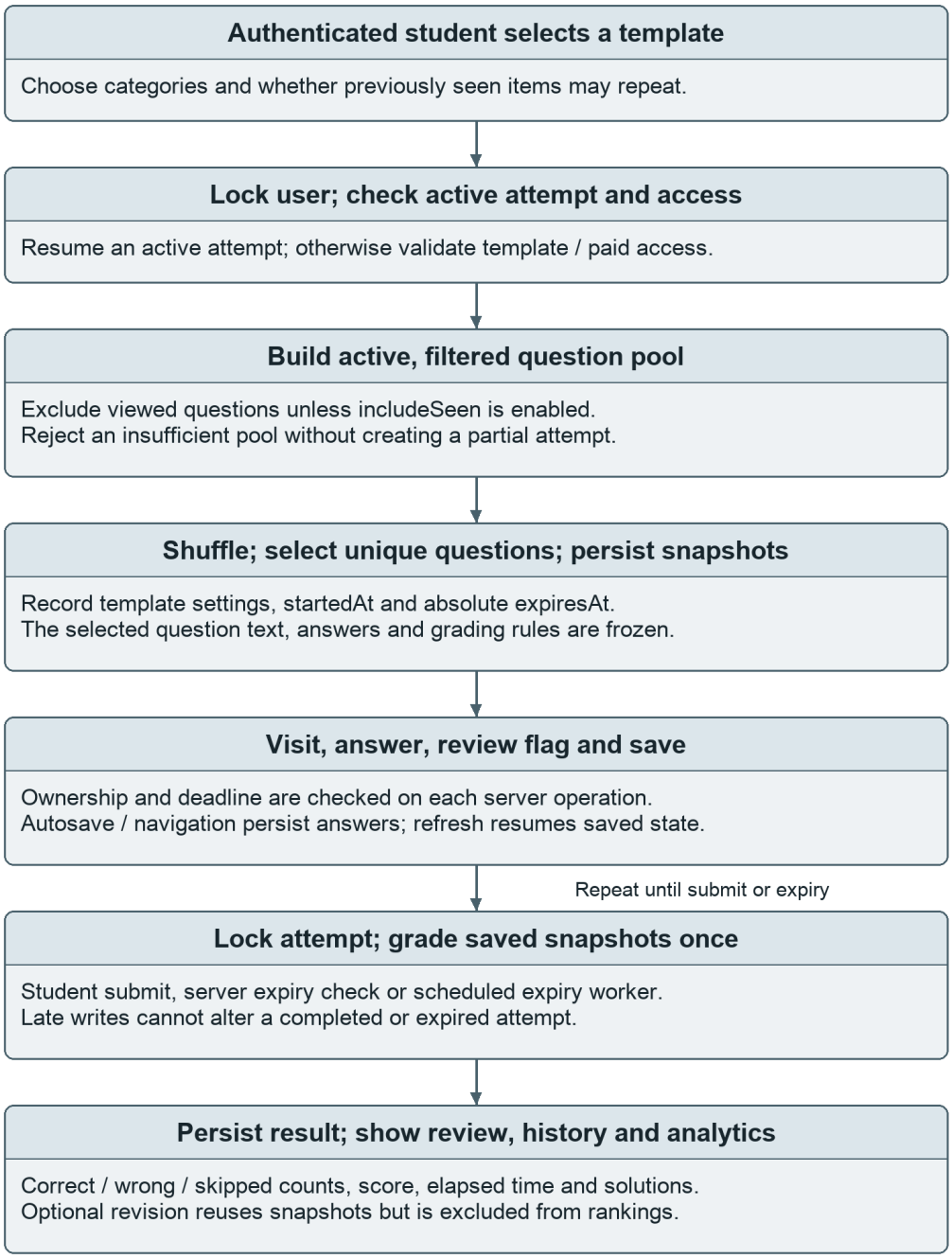


Figure 5.3 Test lifecycle from selection to review.

The lifecycle in Figure 5.3 has one persisted in-progress state and one completed state. Server operations can finalize an expired attempt even if the browser is closed. A repeated start request resumes the existing active attempt instead of creating a parallel practice session.

Source: AttemptService, TestController, ExpiredAttemptJob and app.js [1].

5.9 Timer, autosave, navigation and resume

The practice JSP shows one question, its category/type, an answer control, a review flag, previous/next actions, a numbered navigator, remaining time and submit controls. Single-choice and boolean answers use radios; multiple-choice uses checkboxes; numerical, ratio, fraction and textual types use text input. Passage/data content is rendered as escaped text. Expected answers and explanations are not included in the active-test markup.

The browser counts down from the server-provided absolute expiresAt value. This display is a convenience, not the authority for accepting an answer. AttemptService checks expiry before visiting a question, saving, resuming and submitting. ExpiredAttemptJob also scans up to 100 expired attempts at a configurable 15-second interval and calls autoSubmit. A delayed worker does not extend the deadline because interactive operations enforce the same rule.

Table 5.6 State-saving behaviour

Interaction	Implemented behaviour
Text / choice changes	Text input is debounced by 550 ms; choice changes save immediately through a CSRF-protected same-origin POST.
Save ordering / failure	A Promise chain serializes autosaves; visible status distinguishes saving, saved and failure. Navigation waits for the save.
No JavaScript	Normal Save & next / previous / finish forms still persist answers through server requests.
Refresh / resume	Saved responses and flags are reloaded from the database; a repeated start uses the existing active attempt.
Deadline / submit	Submission locks the attempt, grades saved state once and rejects further changes after completion.

Saving validates the question position within the owned attempt, restricts the number/length of submitted values and checks MCQ option membership. Visiting a question records viewedAt; activity accrues time to the current question, capped by the deadline. This is server-observed interaction time, not a precise measure of attention. Unsent browser edits are not an offline queue: a network failure can still leave the latest keystrokes unsaved.

5.10 Automated evaluation rules

GradingService dispatches on questionType and reads each attempt item's copied answer/configuration JSON. It does not grade an old attempt against the current live question. Null or blank responses are treated as skipped by submission; a non-blank invalid response is incorrect. Each correct response earns one mark, with no negative marking and no partial credit for a partly correct multiple-choice set.

Table 5.7 Grading behaviour for choice and numeric formats

Type	Actual comparison rule
mcq_single	Trim and compare option labels case-insensitively. Save validation also requires a valid option label.
mcq_multiple	Normalize labels to uppercase sets; the complete submitted set must equal the expected set. Order is ignored.
numeric	Parse using BigDecimal after comma removal. Absolute difference must be at most the configured tolerance (default 0.01). Signed decimals and scientific notation are supported.
numeric_with_unit	Apply the numerical comparison. When unit_required is true, the trailing unit must also match case-insensitively; no physical unit conversion is implemented.
data_interpretation	Use the numeric path for a numerical input; otherwise allow a case-insensitive exact textual value. This is not chart understanding or free-form interpretation.

5.10.1 Numeric boundary examples

For an expected value of 10 and tolerance 0.1, both 10.1 and 9.9 lie on the inclusive boundary, whereas 10.11 does not. An expected zero can be answered with "0"; it must not be mistaken for a missing response. If 50 km/h is required, "50 KM/H" may match but "50" and "50 m/s" do not. These examples reflect branches and test cases in the source, not newly measured execution results.

The numeric parser does not serve as a general unit-validation engine: numeric formats can match a numerical prefix without requiring a unit, while numeric_with_unit enforces the suffix only when configured. Administrators must therefore author grading rules appropriate to the intended question.

5.10.2 Fractions, ratios, booleans and text

Table 5.8 Grading behaviour for the remaining supported formats

Type	Actual comparison rule
fraction	Compare cross-products for numerator/denominator equivalence; reject zero denominators. Decimal equivalence is opt-in and uses an absolute 0.01 tolerance.
ratio	Require the same number of colon-separated terms. Compare terms exactly by default; allow proportional scaling only when allow_scaled_equivalent is true.
boolean	Accept true/yes/y/1 or false/no/n/0 case-insensitively. An unrecognized word is invalid, not implicitly false.
fill_in_blank	Compare a value or one of an answers array after Unicode NFKC normalization and configured case, punctuation and whitespace handling.
short_text	Use the same configurable normalization/accepted-alternative comparison as fill_in_blank, not semantic or free-form marking.

Thus 2/4 is equivalent to 1/2 without needing a decimal option, while “0.5” is accepted only when decimal equivalence is enabled. A ratio of 4:6 matches 2:3 only under the scaling option. Boolean false and the string “0” remain valid supplied values. Text defaults are case-insensitive and whitespace-normalized; punctuation is ignored only when configured.

5.10.3 Submission totals and score

$$\text{Score (\%)} = 100 \times \text{correct} / \text{total questions}$$

$$\text{Accuracy (\%)} = 100 \times \text{correct} / (\text{correct} + \text{wrong})$$

The score denominator includes skipped questions. Accuracy excludes skipped items and is zero when nothing was answered. Submission stores correct, wrong and skipped totals, completed status, submittedAt and elapsedSeconds. The displayed attempt percentage is rounded to one decimal place. Elapsed time and late submission time are capped at the original deadline.

Submission is idempotent: once completed, repeated manual or scheduled submission does not grade again. The server never treats the browser’s score, timer or completion flag as authoritative.

5.11 Results, history, revision and analytics

A completed result displays percentage, correct/incorrect/skipped totals, elapsed time and the response, expected answer and explanation for each question. History is paginated and ordered by start time. Student result access first checks ownership; administrator result inspection uses separate administrator-protected routes and rejects an unfinished attempt.

A revision reattempt is allowed only for a completed owned attempt when both stored and current template policies permit it, the template is active and access requirements are satisfied. It copies the previous ordered snapshots rather than drawing new questions. The new row references the original attempt, preserves the original duration/question set and records separate answers/results.

Table 5.9 Analytics and their actual denominators

Metric	Computation / scope
Counted practice	Completed attempts with reattempt=false; in-progress and revision attempts are excluded.
Average score / accuracy	Mean of fresh-attempt percentages / total correct divided by total answered, rounded to one decimal.
Topic performance	Group counted snapshot items by category; report answered/correct totals, accuracy and average time per viewed item.
Streak / practice time	Consecutive completed-practice days through today or yesterday in IST / sum of elapsed seconds converted to minutes.
Rankings	Enabled, verified students with at least one answered fresh-test item; sort by accuracy, correct total, activity or elapsed seconds per answer.
Solved bank	Deduplicate answered items from completed attempts by original question ID; support category/text filters.

Revision attempts remain reviewable but do not raise cumulative student statistics or leaderboard positions. The administrator summary reports question/status/category and student counts plus a 14-day fresh-practice trend. Some dashboard totals and labels have broader scopes than the trend; screenshot values must therefore be read with their metric definitions rather than assumed to share one denominator.

The analytics are calculated in Java over repository results for small educational datasets. Database-side aggregation, caching and large-cohort performance evaluation remain future work.

5.12 Administrator workflows

Administrator functions are protected by /admin/** role rules, not merely by hiding menu items. The overview links to the question review queue, test templates, student inspection and rankings. The /review route is also restricted to administrators. Students cannot assign roles through registration or bypass these boundaries by sending direct form requests.

Table 5.10 Administrative functions and safeguards

Function	Implemented workflow
Question creation / editing	Bind QuestionForm, apply Bean Validation and type-specific service checks, then store normalized fields.
Review queue	Browse pending questions; approval validates the current content; rejection changes status. Already-reviewed rows cannot be reviewed twice.
JSON import	Upload a bounded array; validate each row; mark imported items pending; roll back all inserted rows on failure.
Template management	Create/edit question count, duration and content filters; choose access/reattempt policy; archive without erasing attempts/payments.
Student management	Search names/emails, view summary/topic/history data, inspect completed results and enable/disable student accounts.
Demonstration outbox	Inspect bounded in-memory account-link messages only when demonstration mode is enabled.

5.12.1 Validation and data quality

QuestionService restricts the supported difficulty/type/status values and validates the structure of options, answers and grading configuration. MCQs need 2–12 non-empty labelled options; expected labels must exist. Fractions must have a non-zero denominator, ratio answers need at least two finite numeric terms, boolean answers use JSON booleans, and numeric tolerance cannot be negative. Invalid forms return readable validation messages rather than trusting browser constraints alone.

These checks prevent structural errors but do not prove that the educational content or solution is correct. The 51 pending seed rows remain separate from live practice for review, and the small verbal category requires expansion. Source references and preserved snapshots support traceability, but formal question-version auditing and staff review history are not implemented.

5.13 Payment access and external-service scope

One seeded template, Hard Practice - 60 Qs, is marked paid at ₹50.00. Access is checked on the server through a succeeded PaymentRecord for the current user and template. Free templates require no payment row. A successful local access record unlocks the template, not an unlimited external subscription product.

Table 5.11 Checkout modes in the inspected implementation

Mode	Behaviour and boundary
Demonstration (default)	Create a labelled demo order; confirmation marks the owned order succeeded without making a real charge. Requires demonstration mode.
Disabled	Reject paid checkout and guide the student to a free test. This is the optional MySQL profile's default.
Razorpay (optional)	Create the provider order server-side, verify callback HMAC, fetch payment details and require matching captured payment/order/amount/INR currency. Private keys are needed.

PaymentService fixes the amount from the stored template, associates each order with the authenticated user/template and reuses a compatible pending order. An order cannot be confirmed for another account or template. The signature is HMAC-SHA256 over order_id|payment_id and is compared with MessageDigest.isEqual. Only a valid captured provider response marks a Razorpay order succeeded; a client success message alone is insufficient.

5.13.1 Limits of the payment and mail evidence

Demonstration checkout is an educational convenience and is not evidence of a bank transaction. Source inspection and the repository's signature/ownership tests support the code path, but live Razorpay calls and SendGrid delivery were not verified here or claimed by the repository's test report. Refunds, webhooks, reconciliation and production financial operations are not implemented [2, 3].

MailService similarly supports a bounded demonstration outbox or an outbound SendGrid API call. The default run does not require an external mail account. Before real deployment the operator must disable public demonstration accounts/link previews, configure a verified sender and test delivery failures using private credentials.

5.14 Security, validation and deployment boundaries

Table 5.12 Implemented controls and the limitation each leaves

Control	Implemented evidence / remaining boundary
Authentication and roles	Spring Security session/form login, BCrypt and server route rules; no MFA or login rate limiter is implemented.
Ownership / request integrity	Principal-based user IDs, attempt ownership and CSRF on forms/autosaves; no proctoring or cheating detection is provided.
Session revocation	Credential-version/account-state filter, HttpOnly and SameSite cookies; HTTPS/Secure and trusted proxy setup need deployment configuration.
Output / input handling	JSTL escaping, typed forms, JSON validation and repository binding; this is not a formal penetration-test certification.
Attempt integrity	Snapshots, locks, server expiry and idempotent submit; multi-node operation and load have not been evaluated.
Operational setup	Environment-driven credentials and non-demo startup guards; backups, migrations, auditing and monitoring still need production work.

Normal deployments retain Spring Security's frame denial and default security headers. Preview embedding is a separate explicit opt-in gated by demonstration mode. Browser localStorage is used for theme preference, not as the account/password store. The application's error handling avoids returning database exceptions or stack traces to end users; autosave failures use a small status/error response.

5.14.1 Deployment and maintenance considerations

A laboratory run uses the wrapper, a full JDK and writable persistent data directory. A non-demonstration deployment requires an initial administrator, private database credentials, a correct external base URL and appropriate HTTPS/session settings. The MySQL profile is available but its connectivity and schema rollout must be checked separately. Hibernate update should be replaced with controlled migrations before a production rollout.

The source provides a practical single-application foundation. It does not include rate limiting, comprehensive audit trails, distributed sessions, formal accessibility certification or production-load evidence. These are recorded as future scope rather than silently presented as completed security features [3].

6 RESULTS AND DISCUSSION

6.1 Evidence and verification scope

The results in this chapter combine independently checked source facts, genuine repository screenshots and previously documented execution. These are different evidence categories. No Java build, packaged HTTP suite, provider connection or performance benchmark was rerun while preparing this report. The documented application counts below are attributed to docs/TEST_REPORT.md rather than reported as new executions [2].

Table 6.1 Evidence summary and status

Evidence	Result / status
Seed JSON recount — this report	238 rows: 187 active and 51 pending; ten answer types; active difficulty/category totals reconciled.
Source inspection — this report	Five seeded templates; six mapped entities; role/ownership/expiry/snapshot/grading rules inspected.
Repository Java verification	34 grading/signature cases + 27 integration cases = 61 passed; reported 0 failures/errors/skips. Not rerun here.
Repository packaged acceptance	47 HTTP checks reported passed against the executable WAR and real JSP compilation. Not rerun here.
Repository browser checks	Reported desktop/mobile interaction, a 60% warm-up, save/timer/review/theme checks; not an independent new browser run.
External / nonfunctional checks	No new MySQL/provider/load/security-audit evidence. These remain explicit limitations.

The Java sources contain 31 parameterized grading inputs plus three ordinary grading/signature tests and 27 integration tests, consistent with the documented 61-case total. Integration topics include CSRF and role isolation, ownership, snapshots, expiry/idempotency, unseen filtering, reattempt statistics, import rollback, token handling and foreign-key protection.

The repository documents Java 21.0.8 execution with Java 17 target bytecode, Maven 3.9.12 and a locally provisioned Tomcat/Jasper cache. A stock online JDK 17 build was not separately executed in that environment.

6.2 Authentication interface

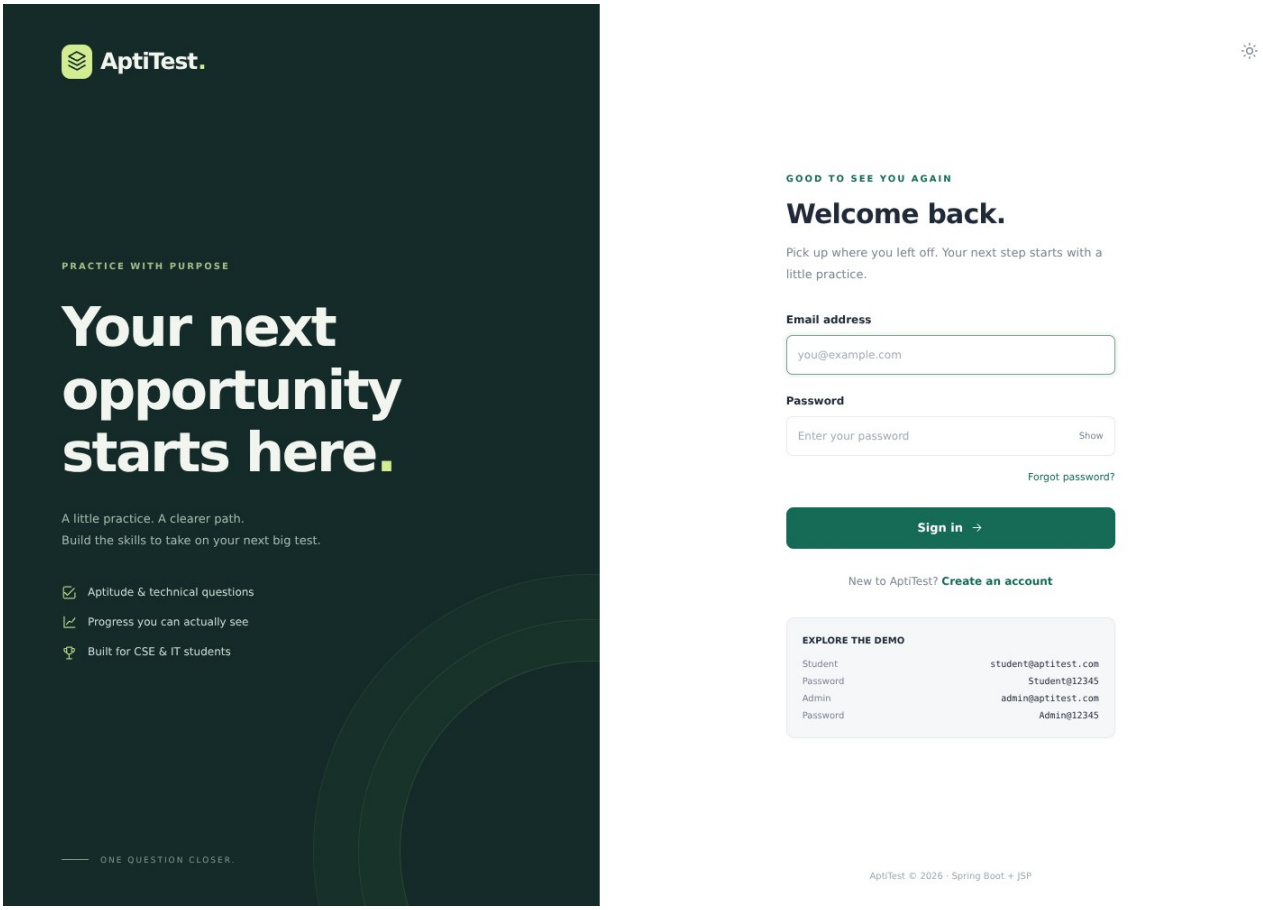


Figure 6.1 Actual AptiTest sign-in page (repository capture, full frame).

The sign-in interface provides email/password fields, a visibility toggle, recovery navigation and account creation. After successful authentication, the same application sends students to their practice workspace and administrators to management. The split-screen appearance is presentation; enforcement is supplied by Spring Security and server-side route rules.

The lower panel contains public demonstration credentials and the capture reflects that configured environment. It is reproduced without disguising the operating mode. Those accounts and link previews must not be carried into a real deployment. The screenshot is evidence of an interface rendered by the application, not proof of production account security.

Provenance: docs/screenshots/01-login.png in source revision d23b6e7 [1, 2]. No generated or reference-project screen is used.

6.3 Student dashboard and test selection

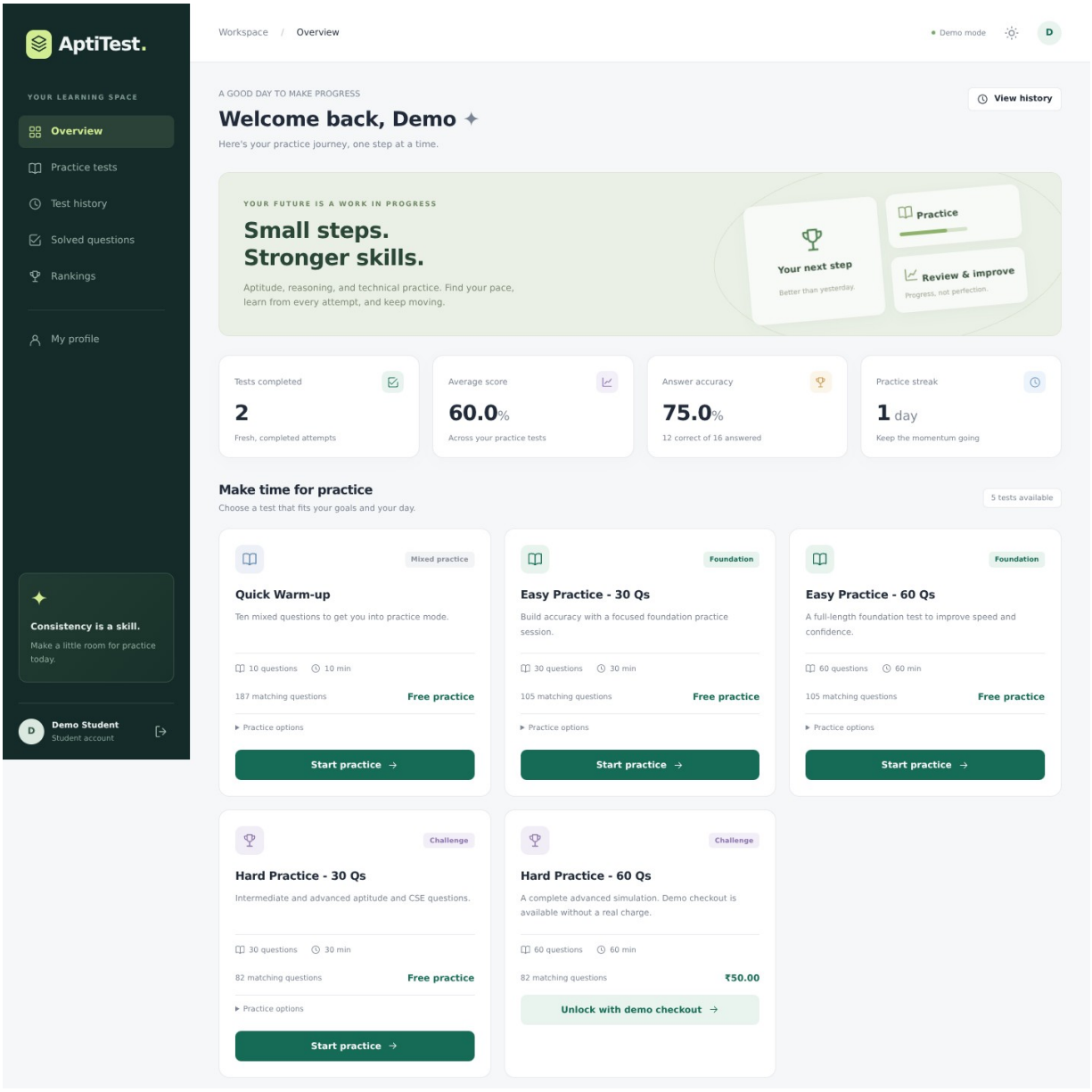


Figure 6.2 Student dashboard and five test cards (upper crop of actual capture).

The dashboard combines recent practice metrics with five template cards, including both 60-question configurations. Pool counts and the paid hard-test card are consistent with the seed/configuration inspection. The visible 60% average and 75% accuracy are values for the captured account, not cohort-level outcomes or measured learning improvement.

Provenance: docs/screenshots/02-student-dashboard.png, cropped to x=0–1440, y=0–1435 for legibility. Values and aspect ratio are unchanged.

6.4 Timed practice and persisted responses

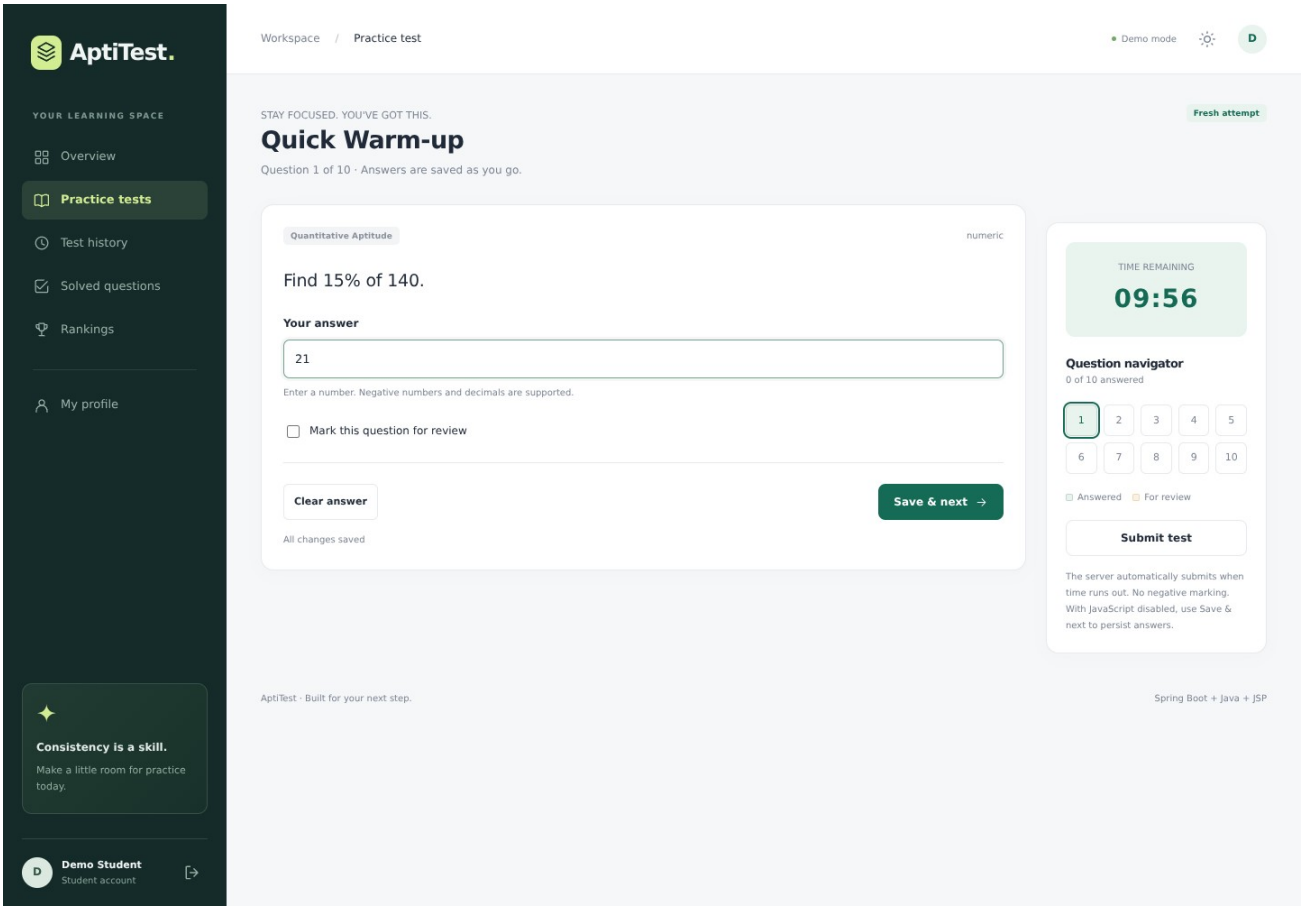


Figure 6.3 Actual practice screen with timer, navigator and save status.

The capture shows a numerical question, the answer 21, a live remaining-time display, review marking, the question navigator and Save & next/Submit controls. The save status is visible below the answer form. Other supported formats use the corresponding radio, checkbox or text controls described in Section 5.9.

The repository reports a completed browser walkthrough and refresh/autosave checks [2]. Source inspection confirms that the display is backed by persisted attempt items and server deadline enforcement. A screenshot alone cannot establish uninterrupted save behaviour, exact timeout timing or every failure path; those claims require executable tests.

Provenance: docs/screenshots/03-practice-test.png (1440 × 1000), full frame. The screenshot is not a newly captured run for this report.

6.5 Result review and interpretation

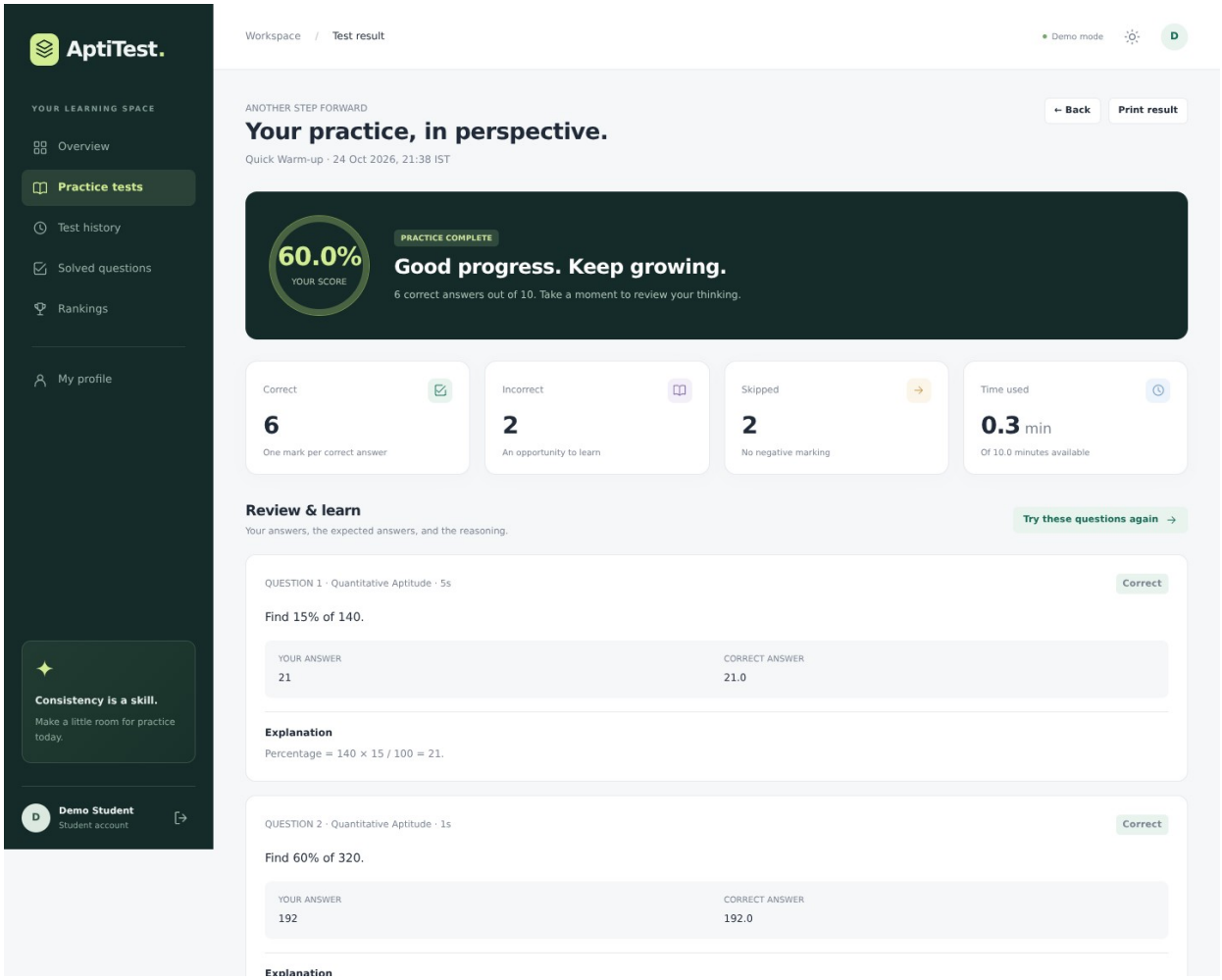


Figure 6.4 Result summary and initial explanations (upper crop of actual capture).

The captured ten-question result contains six correct, two incorrect and two skipped responses. Its displayed 60.0% follows 6/10 × 100; accuracy for that attempt is 6/8 × 100 = 75.0%. The distinction explains why a dashboard can show a higher answer accuracy than score. It is a consistency check of visible data, not a new application-test outcome.

The review exposes the submitted response, stored correct answer and explanation only after completion. Skips remain in the score denominator and do not receive negative marks. The crop preserves the summary and initial review items rather than shrinking the entire long page to unreadable text.

Provenance: docs/screenshots/04-result.png, cropped to x=0–1440, y=0–1150. The complete source capture is retained with the report-generation assets.

6.6 Administrator overview

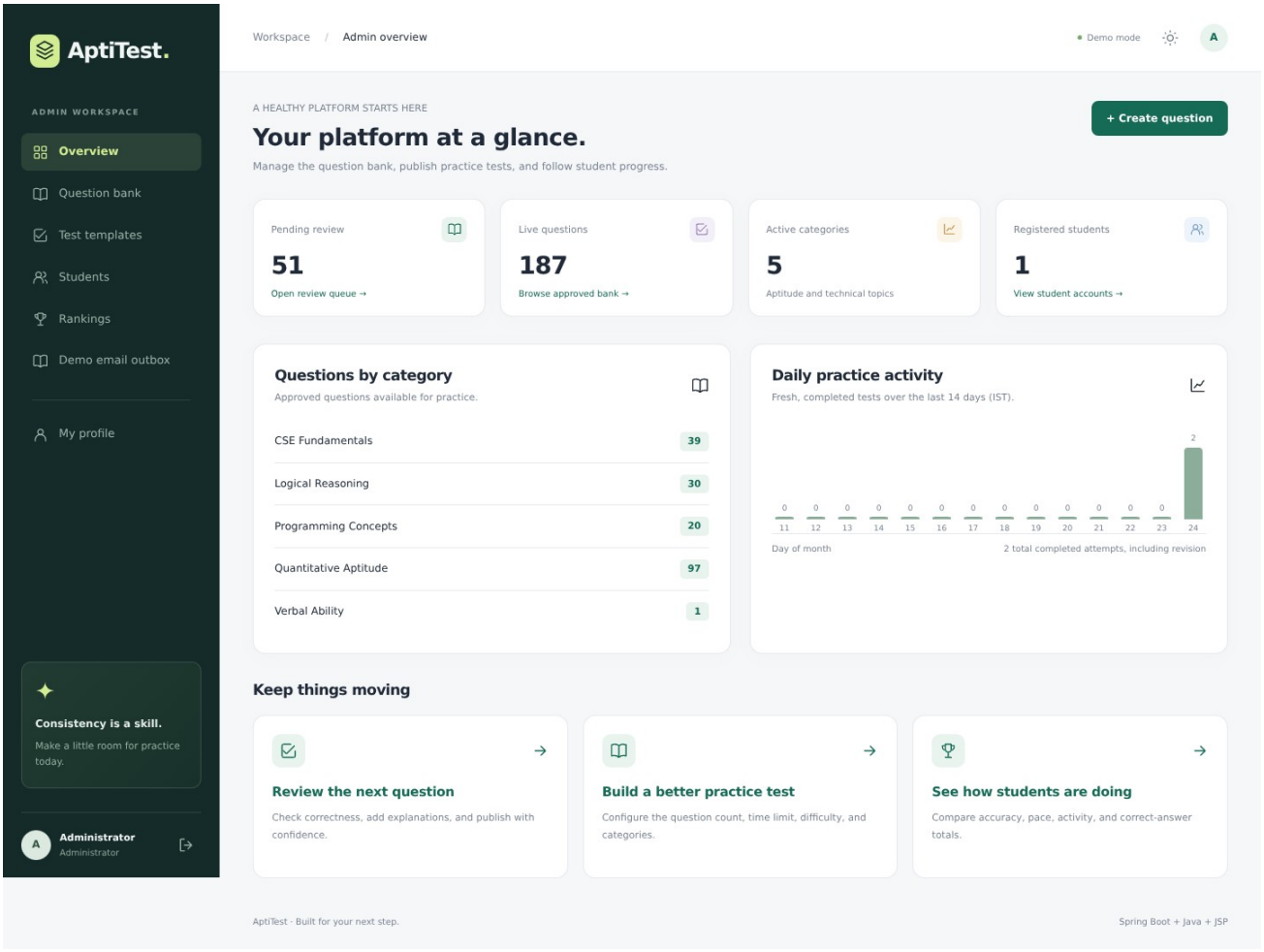


Figure 6.5 Actual administrator overview and content-status summary.

The administrator capture shows 51 pending questions, 187 live questions and five active categories. Its category list reconciles to the independently counted seed data: 39 CSE Fundamentals, 30 Logical Reasoning, 20 Programming Concepts, 97 Quantitative Aptitude and one Verbal Ability item. Registered-student and activity values belong to that captured instance and are not deployment adoption statistics.

The source computes the practice trend from fresh completed attempts, while other totals may include revision. One caption in the supplied overview uses broader wording than that trend's calculation. The report therefore relies on the implementation's metric definitions instead of interpreting every screenshot label as an identical aggregation scope.

Provenance: docs/screenshots/05-admin-dashboard.png (1440 × 1082), full frame. No value, label or chart has been altered.

6.7 Responsive presentation and screenshot provenance

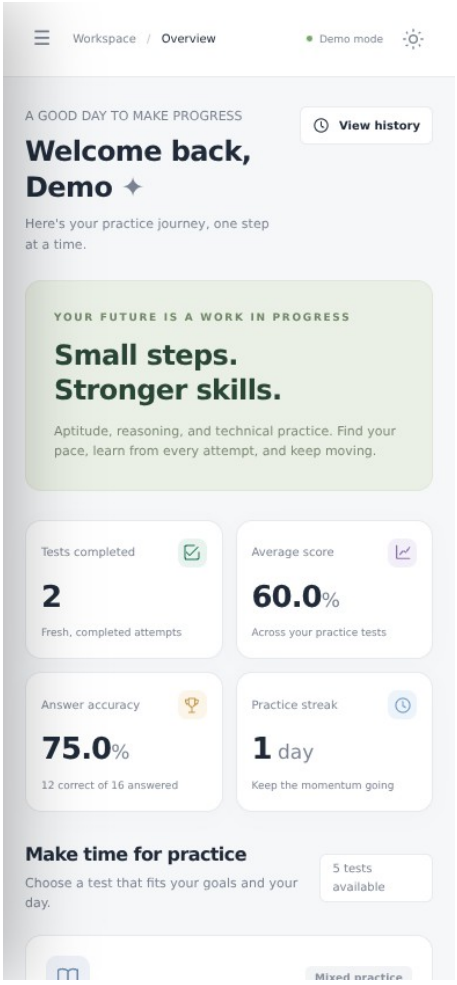


Figure 6.6 Mobile dashboard, first viewport of the genuine 390-pixel-wide capture.

The mobile view stacks metric cards and test content into a narrow layout and provides compact navigation. The repository documents Chromium checks at 390 × 844 and at desktop size without horizontal overflow or uncaught page errors [2]. This supports the stated responsive workflow, but does not constitute a formal accessibility audit or exhaustive cross-browser certification.

All six interface figures originate from docs/screenshots at the inspected Aptitest-JSF revision. Login, practice and administrator views are shown in full. Dashboard, result and mobile figures are explicit rectangular crops, preserving original pixels and proportions. No screenshot was fabricated, repainted, composited with mock metrics or copied from the unrelated presentation reference.

Mobile crop coordinates: x=0–390, y=0–844 from 06-mobile-dashboard.png. Original files and the crop procedure are preserved in report_source/.

6.8 Discussion, reproducibility and limitations

The inspected implementation addresses the principal practice lifecycle: controlled content publication, dynamic selection, persisted timed attempts, type-aware grading and historical review. Snapshots separate historical results from later question edits; server checks and serialized submission reduce inconsistent outcomes; distinct revision handling protects cumulative statistics. These are implementation observations, not a guarantee of correctness under every deployment condition.

Table 6.2 Reproduction procedure and interpretation

Procedure	What it establishes / does not establish
./mvnw clean verify	Compile and run the project's Java tests using a compatible JDK and dependency access. Must be rerun on the evaluator's machine.
java -jar target/aptitest.war	Run the packaged JSP application; use an isolated datasource for verification rather than altering real practice data.
python3 scripts/smoke_test.py --war target/aptitest.war	Execute packaged HTTP/session/CSRF/JSP checks described in the repository. Prior 47-check result is documented, not repeated here.
Configured MySQL / provider tests	Separately validate database rollout, SendGrid and Razorpay failure/success paths with private test credentials; existing screenshots do not cover them.

6.8.1 Known limitations

- Question breadth is uneven: the active verbal category has one item, and 51 seed rows still require editorial review.
- Default seeding, mail previews and no-charge checkout are educational behaviours, not production facilities.
- No controlled production migration, distributed-session setup, load benchmark, proctoring, formal accessibility audit or independent security test is claimed.
- Payment refunds/webhooks/reconciliation, audit history and rate limiting remain outside the implemented scope.

Consequently, the project is suitable for demonstrating a Java/Spring practice platform within the laboratory scope. A real institutional or commercial rollout requires additional operational controls and fresh environment-specific verification.

7 CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

AptiTest integrates the main activities of aptitude practice into one Java/Spring application: account handling, question review, configurable test generation, timed answering, response persistence, automated evaluation and performance review. Its server-rendered JSP/JSTL interface supports separate student and administrator workflows, while the service and repository layers concentrate the assessment rules and persistence responsibilities.

The significant implementation choices are explicit question-type grading, immutable per-attempt content snapshots, a randomized filtered pool, a viewed-question exclusion option, server-side deadlines and idempotent submission. Together they provide a more coherent practice lifecycle than a collection of static papers and manually consolidated result sheets. Revision is treated as a separate learning activity rather than a means of increasing leaderboard totals.

The inspected seed bank contains 238 questions, of which 187 are active and 51 pending, and five default templates. Source inspection and genuine captures support the feature description. The repository reports 61 Java cases and 47 packaged acceptance checks; those results remain attributed to their original verification document and are not presented as rerun evidence. External MySQL, mail, gateway and production-scale behaviour still require separate evaluation.

7.2 Learning outcomes

The mini-project demonstrates practical use of Spring MVC request handling, JSP/JSTL view composition, Spring Security authorization, Bean Validation, transactional services, JPA relationships and Java-based grading logic. It also illustrates why a user-visible timer, a save indicator or a dashboard number needs a clearly defined server-side rule and denominator.

Within its stated educational scope, the system provides a maintainable foundation for structured CSE/IT aptitude practice. Its limitations are actionable: improve content quality, strengthen deployment controls and verify real integrations before broadening the usage claim.

7.3 Prioritized future scope

Table 7.1 Proposed enhancements — not represented as implemented features

Priority	Proposed work	Reason
1 — Content quality	Review pending items; expand verbal/technical pools; add staff review/version history.	Increase breadth and make content changes auditable.
1 — Operational safety	Use versioned migrations, tested backups, private credentials, HTTPS and environment checks.	Replace laboratory defaults with controlled deployment.
1 — Abuse prevention	Add rate limits, stronger account controls, audit logging and independent security review.	Harden the externally exposed account and admin surface.
2 — Integration assurance	Test MySQL and provider paths; add gateway webhooks, reconciliation and refunds.	Make real operational/financial state recoverable and traceable.
2 — Scalability / observability	Move aggregation to database queries; add caching, metrics and shared sessions if scaled.	Support larger datasets and multiple application instances.
2 — Accessibility / quality	Conduct accessibility and browser/device audits; expand automated UI and failure-path tests.	Validate inclusive and dependable interaction, beyond visual responsiveness.
3 — Learning extensions	Evaluate topic-balanced/adaptive selection and richer progress reporting.	Improve learning support only after defining fair rules and measuring outcomes.

These proposals extend the current design but require new implementation and evidence. In particular, adaptive selection, advanced analytics, commercial payment operations and proctored assessment are not existing capabilities. A sensible next iteration should first resolve content review and operational safety before adding new feature breadth.

REFERENCES

[1] MKSubrahmanya12345, “Aptitest-JSF,” source repository, revision d23b6e798b4c82222642ab03a7ba9f8f1b72163c. Primary evidence: pom.xml, application properties, src/main/java, JSP/JavaScript, seed/questions.json and test sources. https://github.com/MKSubrahmanya12345/Aptitest-JSF

[2] Aptitest-JSF, “Verification report,” docs/TEST_REPORT.md, dated 24 October 2026. Repository-documented Java, packaged HTTP and browser verification; original screenshots in docs/screenshots. https://github.com/MKSubrahmanya12345/Aptitest-JSF/blob/d23b6e798b4c82222642ab03a7ba9f8f1b72163c/docs/TEST_REPORT.md

[3] Aptitest-JSF, “Security and implementation boundaries” and “Deployment,” docs/SECURITY.md and docs/DEPLOYMENT.md, same source revision. Configuration, provider and production-scope limitations. https://github.com/MKSubrahmanya12345/Aptitest-JSF/tree/d23b6e798b4c82222642ab03a7ba9f8f1b72163c/docs

[4] Oracle, Java Platform, Standard Edition 17 Documentation. Language and platform reference for the declared Java target. https://docs.oracle.com/en/java/javase/17/

[5] Spring, Spring Boot 3.5 Reference Documentation. Application configuration, dependency management and executable packaging. https://docs.spring.io/spring-boot/3.5/

[6] Spring, Spring Security Reference. Form login, authorization, CSRF and password storage. https://docs.spring.io/spring-security/reference/

[7] Spring, Spring Data JPA Reference. Repository abstraction, persistence and locking. https://docs.spring.io/spring-data/jpa/reference/

References (continued)

[8] Eclipse Foundation, Jakarta Server Pages 3.1 and Jakarta Standard Tag Library 3.0 specifications. https://jakarta.ee/specifications/pages/3.1/ and https://jakarta.ee/specifications/tags/3.0/

[9] H2 Database Engine, Features and Connection Modes. Reference for embedded/file persistence. https://h2database.com/html/features.html

[10] Oracle, MySQL 8.4 Reference Manual. Optional database deployment reference; no MySQL execution result is inferred from this citation. https://dev.mysql.com/doc/refman/8.4/en/

[11] Apache Maven Project, Maven Guides and Maven Wrapper documentation. https://maven.apache.org/guides/ and https://maven.apache.org/wrapper/

[12] JUnit Team, JUnit 5 User Guide. Parameterized and ordinary tests used by the project's Java verification sources. https://docs.junit.org/5.12.2/user-guide/

[13] Razorpay, Standard Checkout: Server Integration and Payment Signature Verification. Reference for the optional provider path. https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/integration-steps/

[14] NMAM Institute of Technology, "JSF_MP_Report_Template_New_2026-27_SEC_B," user-supplied institutional report template, PDF attachment. Source for the cover/certificate identity, chapter order, typography instructions and submission date. The editable original DOCX was not included; the supplied PDF layout was recreated on A4.

Technical claims in the report are grounded in the source revision and its documented verification. External documentation is cited as a real technology reference, not as evidence that an external service was exercised.